SUPERLINEAR CIRCUIT LOWER BOUNDS FOR E^{NP}, ASSUMING THE NEW #SAT ALGORITHMS

VAIBHAV TULSYAN

Abstract. In October 2026 OpenAI and Anthropic claimed deterministic subexponential-time algorithms for #k-SAT, refuting the Exponential Time Hypothesis. Anthropic's manuscript notes that, by a theorem of Williams, this implies that E^{NP} does not have circuits of linear size. We record the quantitative form. Assuming the clause-count bound $2^{C\cdot kM/\sqrt{\log M}}$ of Anthropic's Main Theorem, in the form of a Turing machine with engine constant C, there are $\varepsilon > 0$ and a language $L_0 \in \mathrm{E}^{\mathrm{NP}}$ such that for infinitely many input lengths n, L_0 has no circuit of size $\varepsilon\, n\sqrt{\log n}$ over the full binary basis. The bound proved in OpenAI's Lean development gives $\varepsilon\, n\sqrt{\log n}/(\log\log n)^2$ in the same way. The proof is Williams' argument with the constants tracked: guess a witness circuit, check it with a circuit of $3s + O(n)$ gates, apply Tseitin's transformation to get at most $4S + 1$ clauses, and decide satisfiability with the new algorithm; any $\varepsilon < 1/(36aC)$ works, where a is the input-length shift of the construction. We explain why these algorithms give nothing at $n \log n$, nothing for NEXP or NQP, and nothing more from counting. The constant ε is not explicit, the bound holds only infinitely often, and everything is conditional on the claimed refutations, which are under review.

1. Introduction

Two manuscripts, by OpenAI [11] and by Anthropic [3] (below [O] and [A]), each claim a deterministic algorithm that counts the satisfying assignments of a k-CNF in subexponential time, and hence a refutation of the Exponential Time Hypothesis (ETH). Both come with Lean 4 formalisations of their top-level statements. These are *claimed* results, still under review. The Main Theorem of [A] (Theorem 1, made explicit in its Corollary 11) counts the models of a k-CNF with $m \geq 2$ clauses in which $n \geq 4$ variables occur in time

$$\min\bigl\{2^{C\cdot km/\sqrt{\log m}},\ 2^{C\cdot k^2 n\log\log n/\sqrt{\log n}}\bigr\}$$

for an absolute constant C. Logarithms are to base 2.

Williams [14, Theorem 6.1] showed that if ETH is false, then E^{NP} does not have circuits of linear size; see also Jahanjou, Miles and Viola [6, Corollary 2(1) with Corollary 3(1′)]. [A] draws this consequence and formalises it in Lean (`ENP.CorCircuits`). Its proof uses only the qualitative statement that, for every $\delta > 0$, 3-SAT is decidable in time $2^{\delta N}$. It gives "not $O(n)$" with no rate. Here we use the rate.

1.1. Model.

A *circuit* is over the full binary basis B_2: each gate computes one of the 16 functions $\{0,1\}^2 \to \{0,1\}$ of two earlier wires. Its *size* is the number of gates; inputs are not counted. Circuits are non-uniform, one per input length. For $s\colon \mathbb{N} \to \mathbb{N}$, a language L *has circuits of size s at length n* if some circuit with at most $s(n)$ gates computes $L \cap \{0,1\}^n$. E^{NP} is the class of languages decided in time $2^{O(n)}$ by a deterministic oracle Turing machine with an oracle for SAT, where queries may have length $2^{O(n)}$.

Hypothesis ($\mathrm{H_A}$)**.** *There are a constant $C \geq 1$ and a deterministic multitape Turing machine that, given a 3-CNF ψ with no repeated clause, with $M \geq 2$ clauses and at least 4 occurring variables, encoded in length L, decides whether ψ is satisfiable within $L^C \cdot 2^{3C\cdot M/\sqrt{\log M}}$ steps.*

This is the m-branch of Corollary 11 of [A]with $k = 3$, stated for Turing machines. The constant C here is the *engine constant*, the constant of the machine that is actually run. It is not necessarily the C printed in [A]. [A] counts operations in its own model, and turning those into Turing-machine steps changes C by the polynomial overhead of the simulation. In Anthropic's Lean development the rate is available as the machine

Date: November 2026.

2020 *Mathematics Subject Classification.* 68Q17, 68Q06, 68Q25.

Key words and phrases. Circuit lower bounds, Exponential Time Hypothesis, algorithmic method, E^{NP}, satisfiability algorithms.

form `Tied.TopM` (rate $e_M \le G \cdot k \cdot m/\sqrt{\log(m+2)}$, then a polynomial-time machine on a pad of length 2^{e_M}). The engine constant is then determined by G and by the polynomial exponent of that machine.

Hypothesis ($\mathrm{H_O}$)**.** *There are a constant $C_2 \ge 1$ and a deterministic multitape Turing machine that decides the satisfiability of a 3-CNF with $N \ge 16$ variables, encoded in length L, within $L^{C_2} 2^{C_2 \rho(N)}$ steps, where $\rho(N) = N(\log\log N)^2/\sqrt{\log N}$.*

This is the form of the theorem `exact_count_and_sat` in OpenAI's Lean development, whose rate function is ρ, up to the base of the logarithms. The printed main theorem of [O] has the exponent $O_k(n \log\log n/\sqrt{\log n})$, with one factor of $\log\log n$. It would give the bound of Corollary 1.2 with $(\log\log n)^2$ replaced by $\log\log n$, in the same way.

Theorem 1.1 (Main theorem; machine-checked)**.** *Assume* ($\mathrm{H_A}$)*. There are $\varepsilon > 0$ and $L_0 \in \mathrm{E^{NP}}$ such that for every N there is an $n \ge N$ for which $L_0 \cap \{0,1\}^n$ is computed by no circuit over B_2 with at most $\varepsilon\, n\sqrt{\log n}$ gates.*

More precisely, there is a constant $a \ge 2$, which depends only on the succinct reduction of Imported Theorem 2.2 applied to one fixed language, such that every ε with $36\, a\, C\, \varepsilon < 1$ works.

Corollary 1.2 (Through OpenAI's bound)**.** *Assume* ($\mathrm{H_O}$)*. There are $\varepsilon > 0$ and $L_0 \in \mathrm{E^{NP}}$ such that for infinitely many n, $L_0 \cap \{0,1\}^n$ has no circuit with at most $\varepsilon\, n\sqrt{\log n}/(\log\log n)^2$ gates. Any ε with $3aC_2\varepsilon < 1$ works.*

The n-branch of [A] is a bound in the number of variables, $2^{Ck^2 N \log\log N/\sqrt{\log N}}$. Used in the same way, it gives only $\varepsilon\, n\sqrt{\log n}/\log\log n$. So we use the m-branch.

Remark 1.3 (What the statement does and does not say)*.*

(a) *Infinitely often.* The conclusion is "$\forall N\, \exists n \ge N$". It is not that L_0 fails at a single length. With ε small, $\lfloor \varepsilon n\sqrt{\log n} \rfloor = 0$ for many small n, so a single failing length would be close to vacuous. For a fixed ε, the infinitely-often form cannot be recovered from the single-length form by enlarging the size bound, which is what the linear case does (`ENP.infinitely_often`). We prove it directly by assuming circuits only at lengths $\ge N$. The conclusion does not say that the bound holds for almost every n (Section 5).

(b) *An existential ε.* The constant ε is not explicit, because C is not: [A] and its Lean statement only assert that some C exists. The result therefore does not give $c\, n\sqrt{\log n}$ for every c. That would need a rate $2^{o(M/\sqrt{\log M})}$.

(c) *Strength.* The theorem implies the linear-size statement of [A], and it is strictly more informative: at infinitely many lengths, the lower bound grows faster than every linear function.

1.2. **What is new, and what is not.** The qualitative consequence "$\mathrm{E^{NP}} \not\subseteq \mathrm{SIZE}(O(n))$" is stated, and formalised, in [A]. JMV note that their proofs "provide a close correspondence between the running time for satisfiability and the parameters of the circuit class" [6]. Ben-Sasson and Viola's constant-level instance, as quoted there, is: 3-SAT in time c^n with $c < 2^{1/10}$ implies $\mathrm{E^{NP}} \not\subseteq \mathrm{SIZE}(3n)$. What we add is the substitution of the rate $2^{O(M/\sqrt{\log M})}$, the bookkeeping that keeps every overhead linear, and the resulting explicit superlinear rate. We have not found this rate stated elsewhere. Given how direct it is, it may well have been noticed independently. We claim the careful statement and proof, not the idea.

2. Imported results

Imported Theorem 2.1 (Nondeterministic time hierarchy, Žák [16])**.** *If t_2 is time-constructible and $t_1(n+1) = o(t_2(n))$, then $\mathrm{NTIME}[t_1] \subsetneq \mathrm{NTIME}[t_2]$. In particular, for every $\gamma < 1$ there is a language $L^\star \in \mathrm{NTIME}[2^n] \setminus \mathrm{NTIME}[2^{\gamma n}]$.*

Imported Theorem 2.2 (Local reductions, [6, Theorem 1 and the remark after it])**.** *Let $L \in \mathrm{NTIME}[2^n]$. There are constants d, a_1 and a polynomial-time map sending $x \in \{0,1\}^n$ to a circuit $E_x\colon \{0,1\}^u \to \{0,1\}^{3v+3}$, with $u = n + O(\log n)$ and $v \le a_1 n$, such that the following hold.*

- *$E_x(j)$ lists the three variables (each in $\{0,1\}^v$) and three signs of the j-th clause of a 3-CNF φ_x with 2^u clauses on the variable set $\{0,1\}^v$.*
- *$x \in L$ if and only if φ_x is satisfiable.*

- *Every output bit of E_x is a decision tree of depth $O(1)$ in the bits of j. Hence E_x has at most $d\,n$ gates over B_2.*

JMV state this for algorithms $M(x, y)$ running in time $T \geq 2^{\Omega(n)}$. Their index length is $\ell = \log T + O(\log n)$, and every output bit is a decision tree of depth $O(\log r)$ with $r = n/\log T = O(1)$. Williams' [14, Theorem 6.1] is a *sketch* with the same shape. Its formula comes from a tableau of size $O(2^{3n})$, and its check circuit has "$O(cn)$ wires and $3n + O(1)$ inputs". The explicit count $3s + O(n)$ that we use is the one in JMV's §6 and in Anthropic's Lean module (`StForm`). There the reduction is replaced by a self-contained succinct tableau formula and lazy diagonalisation. We may pad the variable names so that $v = v(n)$ depends only on n and $n \mapsto n + v(n)$ is strictly increasing. Put

$$\ell(n) := n + v(n), \qquad a := \max\{2,\ 1 + a_1\}, \qquad \text{so} \qquad \ell(n) \leq a n.$$

3. Proof of Theorem 1.1

We follow Williams' route [14, Lemma 3.2, Theorem 6.1]. Fix $\varepsilon > 0$ with $36aC\varepsilon < 1$, and fix γ with $36aC\varepsilon < \gamma < 1$. Let $L^\star$ be the language of Imported Theorem 2.1 for this γ, and let $E_x, \varphi_x, u, v, \ell$ be as in Imported Theorem 2.2 for $L = L^\star$. Put

$$s(\ell) := \left\lfloor \varepsilon\, \ell \sqrt{\log \ell} \right\rfloor.$$

3.1. **The language L_0.** For x with φ_x satisfiable, let $w^*(x) \in \{0,1\}^{2^v}$ be its lexicographically first satisfying assignment, indexed by $\{0,1\}^v$. Let L_0 be the set of strings xi with $|x| = n$, $i \in \{0,1\}^{v(n)}$, φ_x satisfiable, and $w^*(x)_i = 1$. Strings whose length is not of the form $\ell(n)$ are not in L_0. Since ℓ is strictly increasing, n and the split xi can be read off from the length.

Lemma 3.1. $L_0 \in \mathrm{E}^{\mathrm{NP}}$.

Proof. This is [14, Lemma 3.2]. On input xi of length $\ell(n)$, write out φ_x by evaluating E_x on all 2^u indices, which takes time $2^{O(n)}$. The language

$$\{\langle \varphi, p \rangle : \varphi \text{ has a satisfying assignment extending the partial assignment } p\}$$

is in NP. Use it to fix the bits of $w^*(x)$ one at a time, in lexicographic order. This takes 2^v queries of length $2^{O(n)}$, after one query that rejects if φ_x is unsatisfiable. Then output bit i. The total time is $2^{O(n)} = 2^{O(\ell)}$. □

3.2. **The algorithm.** Suppose, for a contradiction, that there is an N such that L_0 has circuits of size s at every length $\ell \geq N$. Let n be so large that $\ell(n) \geq N$, and let $x \in \{0,1\}^n$. Consider the following nondeterministic algorithm.

(1) *Guess* a circuit K on $\ell(n)$ inputs with at most $s(\ell(n))$ gates. Hardwire x to get $W(i) := K(xi)$, a circuit on v inputs with at most $s(\ell(n))$ gates (substituting constants does not add gates).

(2) *Build the check circuit* $D\colon \{0,1\}^u \to \{0,1\}$. On input j, compute $E_x(j) = (y_1, \sigma_1, y_2, \sigma_2, y_3, \sigma_3)$. Evaluate the three copies $W(y_1), W(y_2), W(y_3)$. Compute the three literal values $W(y_t) \oplus \neg\sigma_t$ (one gate each), and output 1 if all three are 0 (two gates). So $D(j) = 1$ if and only if clause j of φ_x is violated by the assignment $i \mapsto W(i)$. Its size is

$$S \ \leq\ 3\, s(\ell(n)) + d\, n + 5. \tag{3.1}$$

(3) *Tseitin.* Introduce one variable per input and per gate of D. For a gate $g = f(p, q)$, add for each $(\alpha, \beta) \in \{0,1\}^2$ the clause $(p \neq \alpha) \vee (q \neq \beta) \vee (g = f(\alpha, \beta))$, of width at most 3. Add the unit clause on the output gate. Delete tautological and repeated clauses. The result ψ is a 3-CNF with no repeated clause and

$$M \ \leq\ 4S + 1 \tag{3.2}$$

clauses. It is satisfiable if and only if $D(j) = 1$ for some j, since the gate variables are forced. The number of variables is $u + S$, so also $M \leq 4N_v + 1$, where N_v is the number of variables.

(4) *Decide.* If $M \leq 1$ or fewer than 4 variables occur, decide ψ by brute force. Otherwise run the machine of $(\mathrm{H_A})$. *Accept* if and only if ψ is unsatisfiable.

Lemma 3.2 (Correctness)**.** *For $|x| = n$ as above, the algorithm has an accepting branch if and only if $x \in L^\star$.*

Proof. If $x \in L^\star$, then φ_x is satisfiable. Some circuit K of size $\le s(\ell(n))$ computes L_0 at length $\ell(n)$, so on the branch that guesses it, $W(i) = w^*(x)_i$ for every i. That assignment violates no clause, so $D \equiv 0$, ψ is unsatisfiable, and the branch accepts. If $x \notin L^\star$, then every assignment, in particular $i \mapsto W(i)$ for every guessed K, violates some clause j. So $D(j) = 1$, ψ is satisfiable, and every branch rejects. □

3.3. **The arithmetic.**

Lemma 3.3 (Running time)**.** *For all sufficiently large n, every branch halts within $2^{\gamma n}$ steps.*

Proof. Steps (1)–(3) take $\mathrm{poly}(n, s) = 2^{O(\log n)}$ steps, and the encoding length of ψ is $L = \mathrm{poly}(n)$. It remains to bound the exponent $3C\,M/\sqrt{\log M}$ of step (4).

By (3.1), (3.2) and $\ell(n) \le an$,

$$M \;\le\; M^* := 12\,\varepsilon\,a\,n\sqrt{\log(an)} + 4dn + 21.$$

The function $x \mapsto x/\sqrt{\log x}$ is increasing for $x \ge 2$, since its derivative is $(\log x)^{-1/2}\big(1 - \frac{1}{2\ln x}\big) > 0$ there. Also $M^* \ge n$, so $\log M^* \ge \log n$. Hence

$$\frac{3C\,M}{\sqrt{\log M}} \;\le\; \frac{3C\,M^*}{\sqrt{\log n}} \;=\; 36\,a\,C\,\varepsilon\,n\sqrt{\frac{\log(an)}{\log n}} + \frac{3C(4dn+21)}{\sqrt{\log n}} \;=\; 36\,a\,C\,\varepsilon\,n + o(n). \tag{3.3}$$

So the running time is at most $2^{O(\log n)} \cdot L^C \cdot 2^{(36aC\varepsilon + o(1))n}$, which is at most $2^{\gamma n}$ for large n, because $36aC\varepsilon < \gamma$. □

Proof of Theorem 1.1. The algorithm needs to know $s(\ell(n))$, which is computable in time $\mathrm{poly}(n)$. Under the assumption, Lemmas 3.2 and 3.3 decide $L^\star$ correctly within $2^{\gamma n}$ steps on all inputs of all sufficiently large lengths. The finitely many remaining inputs are handled by a table. Hence $L^\star \in \mathrm{NTIME}[2^{\gamma n}]$, which contradicts Imported Theorem 2.1. So for every N there is an $\ell \ge N$, indeed one of the form $\ell(n)$, at which L_0 has no circuit with at most $s(\ell)$ gates. Together with Lemma 3.1, this is the theorem. □

Remark 3.4 (The input-length shift and the constant).

(a) *The shift.* The witness circuit is a circuit for L_0 at length $\ell(n)$, not n. So the check circuit has $3s(\ell(n)) + O(n)$ gates, not "$3s(n) + O(n)$". In JMV's own construction the E^{NP} input is (x, i) of length $2n + O(\log n)$. For $s = \varepsilon n\sqrt{\log n}$ the shift costs only the factor a in the condition on ε.

(b) *Lean normalisation.* Anthropic's module avoids the hierarchy theorem. It takes m to be the input length of its E^{NP} language. The tableau formula there has time 2^{b_t} with $b_t \ge 3m/8$, and the engine is allowed a budget of $2^{m/32}$ inside a total of $2^{m/8}$ (`exists_threshold`). Its check formula has $N_v \le 3|K| + a_0(m+1) + g(e)$ variables and at most $4N_v + 1$ clauses. With $|K| \le \varepsilon(m+1)\sqrt{\log(m+2)}$, the computation (3.3) becomes $36C\varepsilon m(1 + o(1)) + O(m/\sqrt{\log m}) \le m/32 - o(m)$. This holds for $\varepsilon < 1/(1152\,C)$; for instance $\varepsilon = 1/(1200C)$.

(c) *Which C.* In either normalisation, C is the effective engine constant of $(\mathrm{H_A})$. In Lean this is G together with the polynomial exponent of the machine on the pad, and possibly the exponent of the step bound in `PaddedDescent.exists_tm'`. It is fixed before the formula table is chosen, so ε is still an absolute constant.

Proof of Corollary 1.2. Take $s(\ell) = \lfloor \varepsilon\ell\sqrt{\log \ell}/(\log\log \ell)^2 \rfloor$ and $3aC_2\varepsilon < \gamma < 1$. Run the same algorithm with the machine of $(\mathrm{H_O})$ in step (4). Now $N_v \le u + S \le N^* := 3\varepsilon an\sqrt{\log(an)}/(\log\log n)^2 + (d+2)n$ for large n. The function ρ is increasing for large arguments. We have $\log N^* = \log n + O(\log\log n)$, and so $\log\log N^* = \log\log n + o(1)$. Therefore

$$C_2\,\rho(N_v) \le C_2\,\rho(N^*) = 3aC_2\varepsilon\,n(1 + o(1)) + O\!\left(\frac{n(\log\log n)^2}{\sqrt{\log n}}\right) \le (\gamma - o(1))\,n,$$

and the rest is unchanged. □

4. Comparison with what is known

The following statements were checked against the primary sources.

- E^{NP} *and* P *over* B_2. Before 2026 the best lower bound for a language in E^{NP} was $3.1n - o(n)$, by Li and Yang [8]. They prove it for affine dispersers of sublinear dimension, which are constructible in polynomial time, so the bound holds already in P. It improves the $(3 + \frac{1}{86})n - o(n)$ of Find, Golovnev, Hirsch and Kulikov [4], which in turn improves the $3n - o(n)$ of Blum. Williams wrote in 2010 that "linear size circuit lower bounds are not known for this class" [14], that is, no proof that linear size does not suffice.
- *Above* E^{NP}. Chen, Hirahara and Ren [1] prove that $\mathrm{S_2E}/_1 \not\subseteq \mathrm{SIZE}[2^n/n]$. This gives the same near-maximum bounds for $\Sigma_2\mathrm{E}$, $(\Sigma_2\mathrm{E} \cap \Pi_2\mathrm{E})/_1$ and $\mathrm{ZPE}^{\mathrm{NP}}/_1$. The almost-everywhere, advice-free bound $\mathrm{S_2E} \not\subseteq \text{i.o.-}\mathrm{SIZE}[2^n/n]$ is due to Li [9]. $\mathrm{E}^{\mathrm{NP}} \subseteq \mathrm{S_2E}$, so these classes lie above the one treated here.
- *Fixed-polynomial bounds.* Kannan [7] showed that for each k, $\Sigma_2\mathrm{P} \cap \Pi_2\mathrm{P} \not\subseteq \mathrm{SIZE}(n^k)$. Whether $\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(n^k)$ for some fixed $k > 1$ is open. Before the refutations, so was $\mathrm{E}^{\mathrm{NP}} \not\subseteq \mathrm{SIZE}(O(n))$.
- NEXP *and linear size.* No superlinear lower bound for NEXP follows. JMV raise as an open problem the easy-witness strengthening of [5] to linear-size circuits [6].

So, conditionally, Theorem 1.1 is a superlinear lower bound with an explicit rate for a class below $\mathrm{S_2E}$ over B_2, holding infinitely often. It is the first we know of, but we cannot rule out that it has been stated elsewhere.

5. What these algorithms cannot give

- *Beyond* $n\sqrt{\log n}$. The check circuit and Tseitin's transformation are already linear. So the only lever is the rate: an algorithm in time $2^{M/g(M)}$ converts into $s = \varepsilon n\, g(n)$. Size $n \log n$ would need 3-SAT in $2^{O(M/\log M)}$, and size $n^{1+\delta}$ would need $2^{M^{1-\delta'}}$. Neither manuscript claims such a rate.
- NEXP *and* NQP. Moving below E^{NP} needs an easy-witness lemma. Those of Impagliazzo, Kabanets and Wigderson [5] and of Murray and Williams [10] work for polynomial size. The second loses a polynomial in the size. After the loss, one needs satisfiability of $\mathrm{poly}(n)$-size circuits in time $2^n/n^{\omega(1)}$, and $2^{O(s/\sqrt{\log s})}$ with $s = n^k$ gives nothing. Linear-size easy witnesses are JMV's open problem.
- *Counting adds nothing here.* The NP oracle of E^{NP} is used only to fix the lexicographically first witness, and the final check is a decision question. The counting form of the algorithms is not used. The lower bounds of the algorithmic method that do exploit counting or fast evaluation, such as those against ACC of Williams [15] and Murray–Williams [10], concern polynomial-size circuits and run into the previous item.
- *Almost everywhere.* Chen, Lyu and Williams [2] obtain almost-everywhere lower bounds for E^{NP} (against ACC^0) from non-trivial algorithms. Whether their machinery keeps the check circuit at $O(s)$ gates, so that it would give an almost-everywhere version of Theorem 1.1, we have not checked. Their reductions involve PCPs, whose polylogarithmic overhead would absorb the $\sqrt{\log n}$.
- *Restricted classes.* The algorithms are CNF algorithms, and the reduction goes through general circuits. The implied bounds for formulas or for log-depth circuits are no better than what Theorem 1.1 gives, and for formulas they are weaker than known bounds.

6. Formalisation status

Anthropic's Lean development [3] proves the linear-size form,

$$\texttt{ENP.CorCircuits}: \ \exists L,\ \texttt{InENP}\ L \wedge \neg\, \texttt{HasLinearCircuits}\ L,$$

together with the infinitely-often variant `CorCircuitsInf`. Its circuits are over B_2, and size is the number of gates. E^{NP} is formalised with Mathlib `FinTM2` oracle machines. The proof follows the route above, using the δ-form of the engine (time $c \cdot 2^{\delta N}$ for every $\delta > 0$) and lazy diagonalisation in place of Imported Theorem 2.1. The statement `HasCircuits` $L\ s$ is already parametric in s, but the module hard-wires both the size $c_0(m+1)$ and the time $2^{\delta N}$.

Theorem 1.1 is formally verified in Lean 4 on top of that module, in the public repository [13] (directory `lean-bridge-anthropic/bridge/EthFollowup/Circuits`, five files, 707 lines). The statement is

$$\texttt{enp_not_size_n_sqrt_log}: \quad \exists L\, \varepsilon,\ 0 < \varepsilon \wedge \texttt{ENP.InENP}\ L \wedge \forall N\, \exists n,\ N \le n \wedge \neg\, \texttt{ENP.DecidedAt}\ L\, \lfloor \varepsilon \cdot n \cdot \sqrt{\log_2 n} \rfloor\, n,$$

together with the variants `enp_not_hasCircuits_n_sqrt_log` (the negation of the module's `HasCircuits`) and `enp_not_size_n_sqrt_log_shifted`. Here `ENP.InENP`, `ENP.DecidedAt` and the circuit model are Anthropic's own definitions, and L is their language `CircuitLowerBound.L0`. The theorems depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`, and a fresh `leanchecker` replay of the full dependency closure passed. The three non-arithmetic points of the plan were resolved as follows: the rate solver is packaged as one padded polynomial-time function and turned into a Turing machine by Anthropic's own padded-host lemma, so no polynomial state bound is needed; the solver's count over the occurring variables is compared with zero directly; and the infinitely-often form is obtained by threading the threshold through the module's diagonalisation. The constant is $\varepsilon = 1/(4608(qG+1))$, where q is the host's polynomial degree and G the constant of the rate solver; it is existential because q and G are. Corollary 1.2 (the OpenAI form) is proved on paper only and is not claimed as machine-checked.

7. Limitations

(a) *Conditional.* Everything is conditional on ($\mathrm{H_A}$), or on ($\mathrm{H_O}$) for Corollary 1.2. These are Turing-machine forms of the claimed theorems of [A] and [O]. Expert review of the manuscripts is pending, and if the hypotheses fail, the results lose their proofs. Our proofs are on paper, written with AI assistance, and not yet checked by a human.

(b) *ε is not explicit.* It depends on the effective engine constant C, which is existential in [A] and in its Lean statement, and on the constant a of the reduction. No numerical value of ε is claimed. The bound gives no meaningful statement at any concrete input length.

(c) *Infinitely often only.* The lower bound holds at infinitely many lengths, not at almost every length (Section 5).

(d) *Imported reduction.* Imported Theorem 2.2 is used as stated by JMV, with the linear gate count read off from depth-$O(1)$ decision trees. Williams' Theorem 6.1, which we follow, is itself a sketch. The self-contained version is Anthropic's Lean module, for the linear case.

(e) *Literature.* The statements in Section 4 and all citations were checked against the primary sources (publisher records, arXiv, ECCC, the author's version of [14]) on 4 November 2026. We have not searched systematically for prior statements of Theorem 1.1.

Acknowledgements

This work was done with AI assistance (Claude Code, Anthropic): the observation, the proofs, the referee check of the first draft and the literature checks were produced in sessions with Claude Code. Everything here has been checked against the primary sources as described, but awaits human expert review. Responsibility for any errors rests with the author. This note continues the project of [12].

References

1. Lijie Chen, Shuichi Hirahara, and Hanlin Ren, *Symmetric exponential time requires near-maximum circuit size*, Proc. 56th Annual ACM Symposium on Theory of Computing (STOC), 2024, ECCC TR23-144; arXiv:2309.12912, pp. 1990–1999.
2. Lijie Chen, Xin Lyu, and R. Ryan Williams, *Almost-everywhere circuit lower bounds from non-trivial derandomization*, Proc. 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS), 2020, ECCC TR20-150, pp. 1–12.
3. Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, Preprint dated October 26, 2026. Paper and Lean formalisation in the repository `https://github.com/anthropics/formal-math`, folder `eth`, 2026.
4. Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch, and Alexander S. Kulikov, *A better-than-3n lower bound for the circuit complexity of an explicit function*, Proc. 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS), 2016, pp. 89–98.
5. Russell Impagliazzo, Valentine Kabanets, and Avi Wigderson, *In search of an easy witness: exponential time vs. probabilistic polynomial time*, J. Comput. System Sci. **65** (2002), no. 4, 672–694.
6. Hamid Jahanjou, Eric Miles, and Emanuele Viola, *Local reductions*, Automata, Languages, and Programming (ICALP 2015), Part I, Lecture Notes in Comput. Sci., vol. 9134, Springer, 2015, Numbering as in arXiv:1311.3171. Journal version: "Local reduction", Inform. and Comput. **261** (2018), 281–295, pp. 749–760.
7. Ravi Kannan, *Circuit-size lower bounds and non-reducibility to sparse sets*, Inform. and Control **55** (1982), no. 1–3, 40–56.
8. Jiatu Li and Tianqi Yang, *$3.1n - o(n)$ circuit lower bounds for explicit functions*, Proc. 54th Annual ACM Symposium on Theory of Computing (STOC), 2022, ECCC TR21-023, pp. 1180–1193.
9. Zeyong Li, *Symmetric exponential time requires near-maximum circuit size: simplified, truly uniform*, Proc. 56th Annual ACM Symposium on Theory of Computing (STOC), 2024, arXiv:2310.17762, pp. 2000–2007.

10. Cody Murray and Ryan Williams, *Circuit lower bounds for nondeterministic quasi-polytime: an easy witness lemma for NP and NQP*, Proc. 50th Annual ACM Symposium on Theory of Computing (STOC), 2018, Journal version: SIAM J. Comput. **49** (2020), no. 5, STOC18-300–STOC18-322, pp. 890–901.
11. OpenAI, *The Exponential Time Hypothesis is false*, Preprint dated 17 October 2026, revised 25 October 2026. Paper and Lean formalisation in the repository https://github.com/openai/math, 2026.
12. Vaibhav Tulsyan, *Counting Hamiltonian cycles in sparse graphs in subexponential time, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2610.00672v1, October 2026; source and Lean development at https://github.com/xennygrimmato/eth-hamiltonicity, 2026.
13. ———, *eth-hamiltonicity: Lean 4 formalisation and research notes*, Public repository https://github.com/xennygrimmato/eth-hamiltonicity, Lean sources as of commit https://github.com/xennygrimmato/eth-hamiltonicity/tree/ca17a3762bb21ddbe5942eeac9f29fa58123bb42, 2026.
14. Ryan Williams, *Improving exhaustive search implies superpolynomial lower bounds*, SIAM J. Comput. **42** (2013), no. 3, 1218–1244, Conference version in STOC 2010. Theorem numbers as in the author's version https://people.csail.mit.edu/rrw/improved-algs-lbs2.pdf.
15. ———, *Nonuniform ACC circuit lower bounds*, J. ACM **61** (2014), no. 1, Art. 2, 1–32.
16. Stanislav Žák, *A Turing machine time hierarchy*, Theoret. Comput. Sci. **26** (1983), no. 3, 327–333.